



국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론



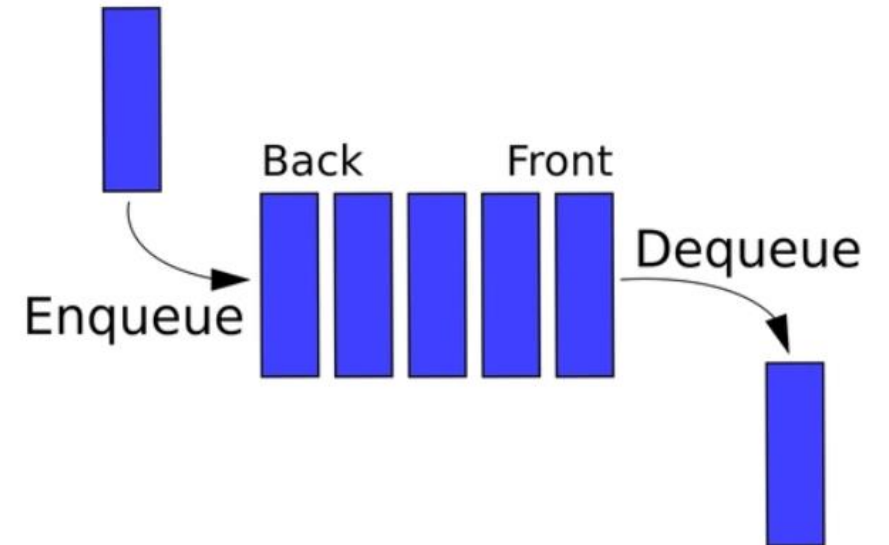
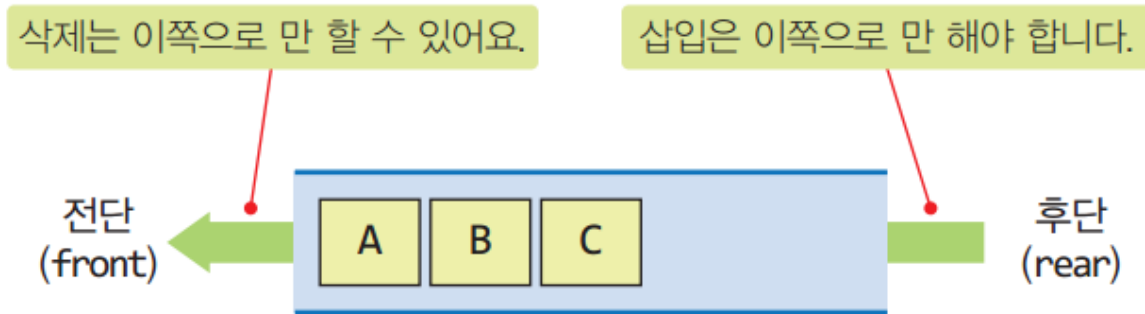
큐

- Queue라는 단어 자체가 표 같은 것을 구매하기 위해 줄서는 것을 의미
- 삽입과 삭제가 양 끝에서 각각 수행되는 자료구조
- 선입 선출(First-In First-Out, FIFO) 원칙 하에 item의 삽입과 삭제 수행
- 먼저 들어온 데이터가 먼저 처리됨
- 일상생활의 관공서, 은행, 우체국, 병원 등에서 번호표를 이용한 줄서기가 대표적인 큐



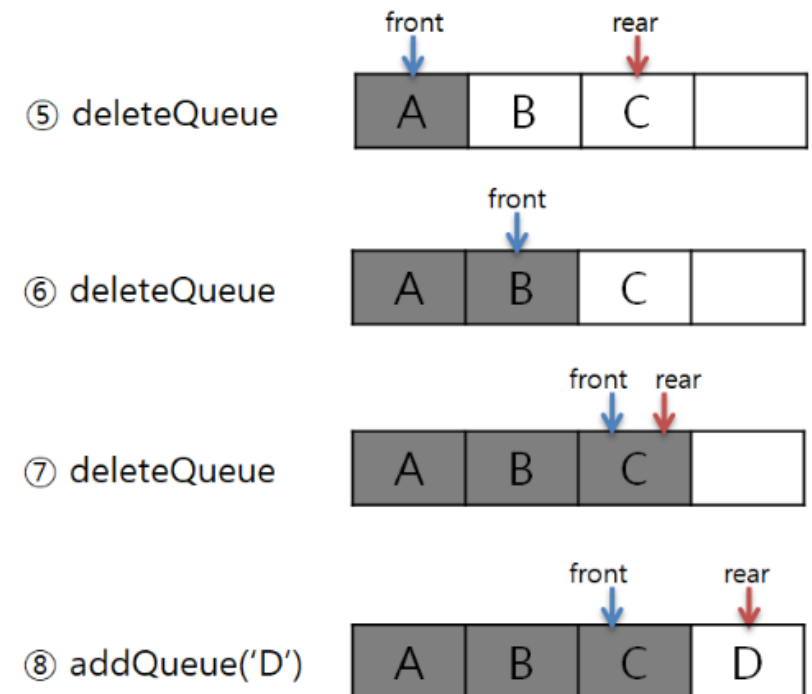
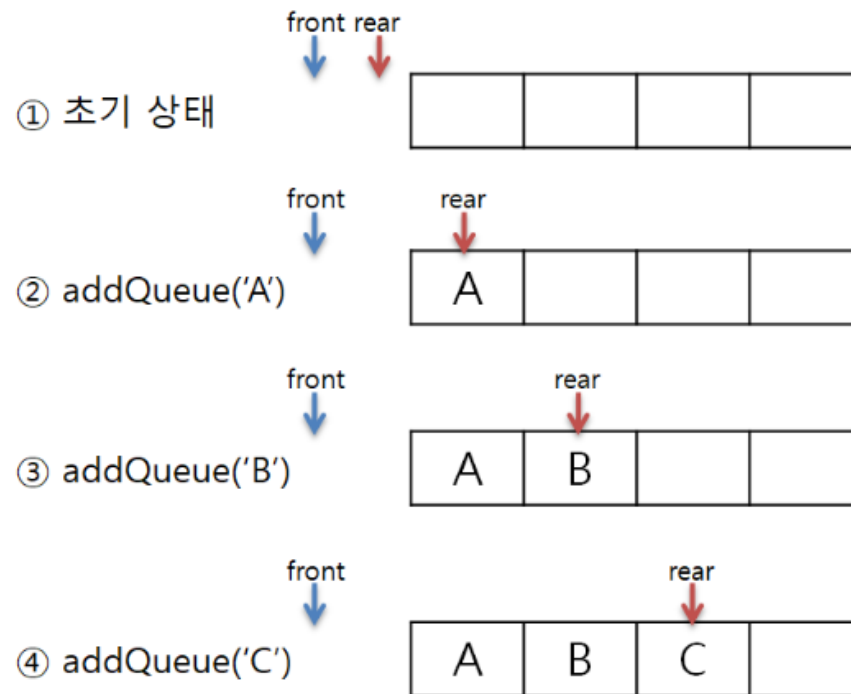
큐의 구조

- 데이터가 들어오는 위치는 가장 뒤(Rear 또는 Back이라고 한다.)에 있고, 데이터가 나가는 위치는 가장 앞(Front라고 한다.)에 있어서, 먼저 들어오는 데이터가 먼저 나가게 된다.
- 입력 동작은 Enqueue, 출력 동작은 Dequeue라고 한다.
- 유일하게 접근 가능한 원소는 가장 먼저 삽입한 원소



Queue의 enqueue, dequeue 연산

- ① createQueue(4);
- ② addQueue(queue, 'A');
- ③ addQueue(queue, 'B');
- ④ addQueue(queue, 'C');
- ⑤ deleteQueue(queue);
- ⑥ deleteQueue(queue);
- ⑦ deleteQueue(queue);
- ⑧ addQueue(queue, 'D');



추상 자료형이란?

- 프로그래밍 언어에서 다양한 자료형 제공
예) 파이썬: 정수(int), 실수(float), 문자열(str) 등
35: int, 35.0: float, "35": str
- 새로운 자료형이 필요하면?
그 자료형의 자세하고 복잡한 내용 대신에 필수적이고 중요한 특징만 골라서 단순화시키는 작업이 필요
→ 추상화(abstraction) → 추상 자료형
- 추상 자료형(ADT: Abstract Data Type)
그 자료형이 **어떤 자료를 다루고, 어떤 연산이 필요한지**를 정의해 보는 것 정도로 이해하면 됨

큐의 추상 자료형

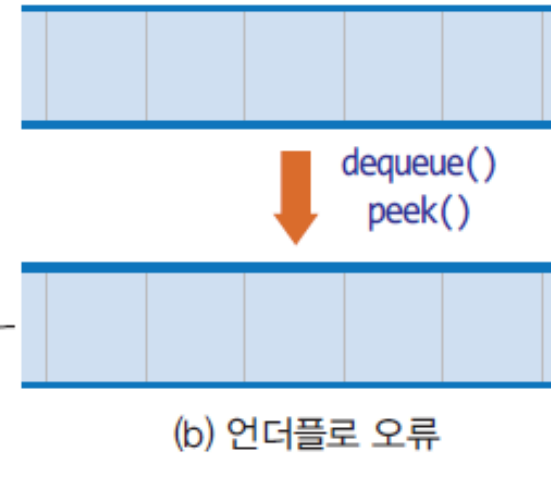
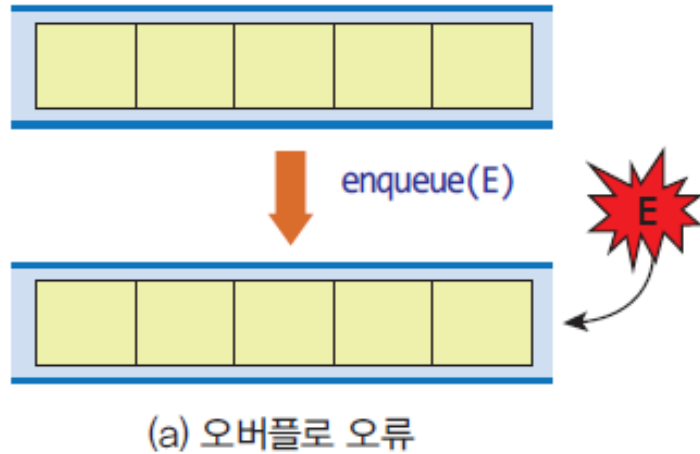
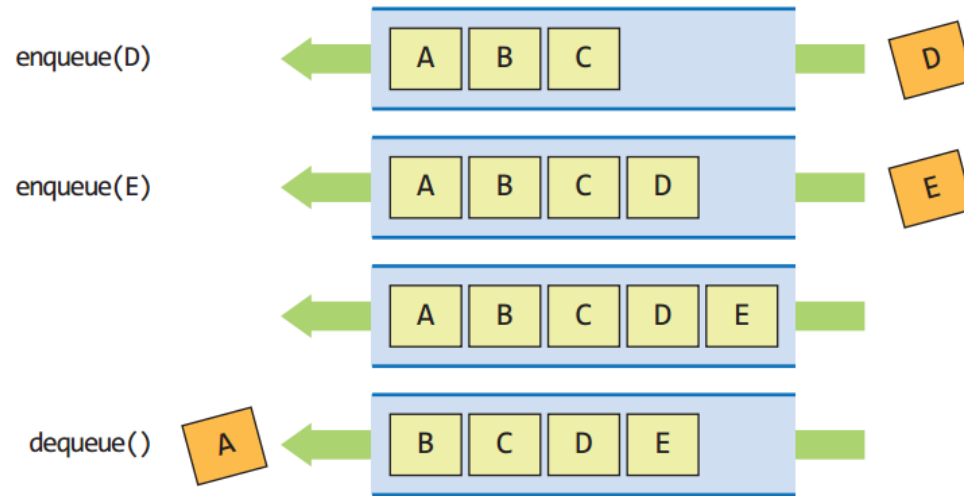
- 큐가 다루는 데이터
 - 숫자와 문자열을 포함한 어떤 자료든 저장할 수 있음
- 큐의 연산
 - 핵심적인 연산들만 관심있음

큐의 ADT

데이터: 선입선출(FIFO)의 접근 방법을 유지하는 항목들의 모임
연산

- `Queue()`: 비어 있는 새로운 큐를 만든다.
- `isEmpty()`: 큐가 비어있으면 `True`를 아니면 `False`를 반환한다.
- `enqueue(x)`: 항목 `x`를 큐의 맨 뒤에 추가한다.
- `dequeue()`: 큐의 맨 앞에 있는 항목을 꺼내 반환한다.
- `peek()`: 큐의 맨 앞에 있는 항목을 삭제하지 않고 반환한다.
- `size()`: 큐의 모든 항목들의 개수를 반환한다.
- `clear()`: 큐를 공백상태로 만든다.

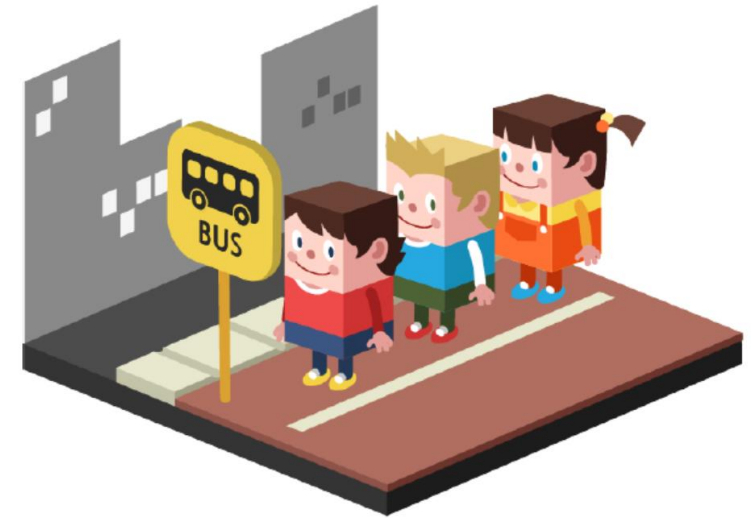
큐의 연산 및 2가지 오류 상황



일반적인 언어에서 배열은 크기를 고정시키고 생성함

큐의 활용 예

- 프린터와 컴퓨터 사이의 인쇄 작업 큐 (버퍼링)
- 실시간 비디오 스트리밍에서의 버퍼링
- 통신에서의 데이터 패킷들의 모델링에 이용
- CPU의 태스크 스케줄링(Task Scheduling)
- 콜 센터의 전화 서비스 처리
- 이진트리의 레벨순회(Level-order Traversal)
- 그래프에서 너비우선탐색(Breath-First Search)
- 기수정렬에서 레코드의 정렬을 위해 사용
- 실시간(Real-time) 시스템의 인터럽트(Interrupt) 처리
- 다양한 이벤트 구동 방식(Event-driven) 컴퓨터 시뮬레이션
- 시뮬레이션의 대기열(공항의 비행기들, 은행에서의 대기열)
- 등



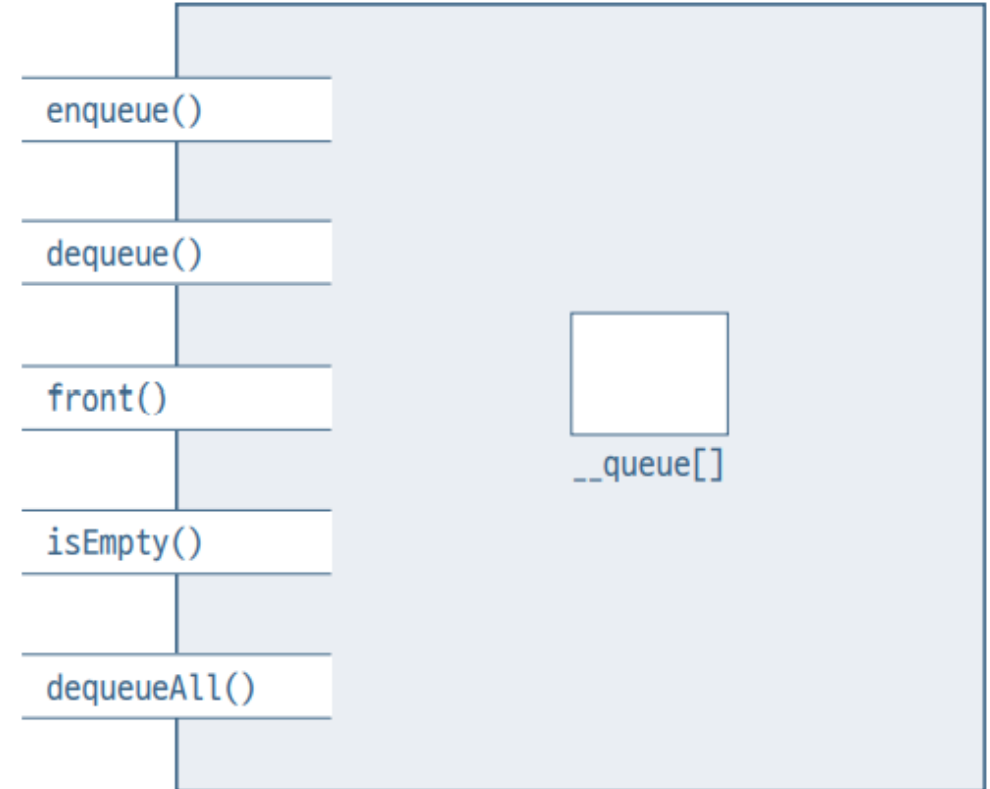
리스트 큐 객체 구조

필드:

`__queue[]` ◀ 큐 원소들이 저장되는 연결 리스트

작업:

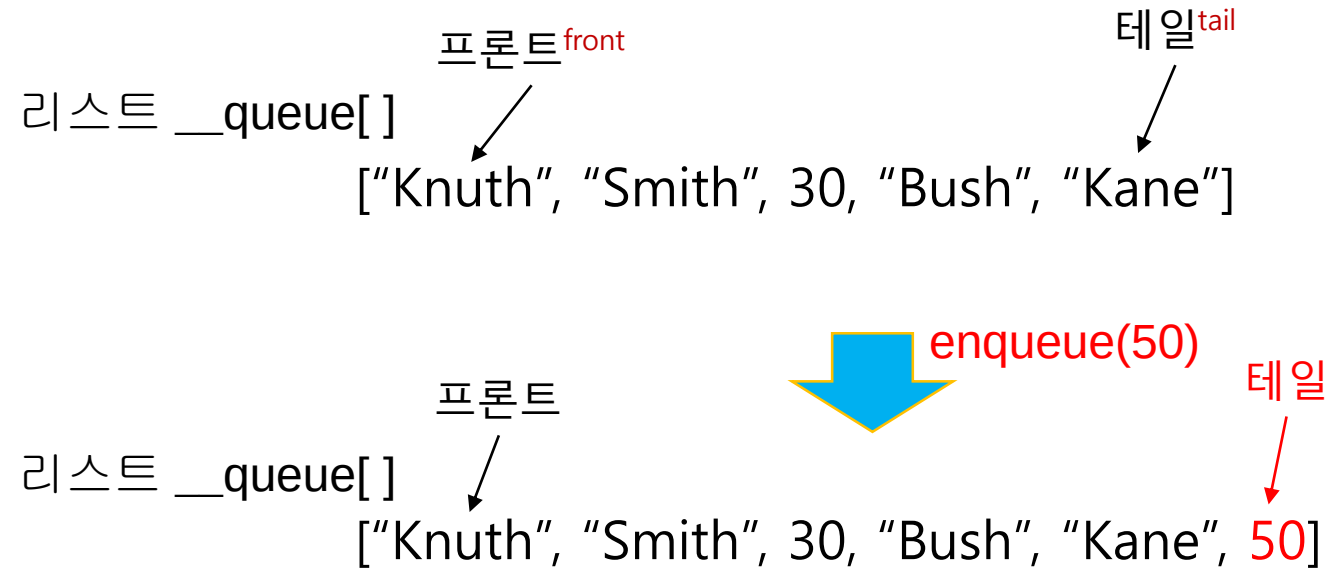
`enqueue()` ◀ 큐의 맨 뒤에 원소를 삽입한다
`dequeue()` ◀ 큐의 맨 앞에 있는 원소를 알려주고 삭제한다
`front()` ◀ 큐의 맨 앞에 있는 원소를 알려준다
`isEmpty()` ◀ 큐가 빈 큐인지를 알려준다
`dequeueAll()` ◀ 큐를 깨끗이 청소한다



리스트를 이용한 큐 객체 구조

삽입(insertion) : enqueue

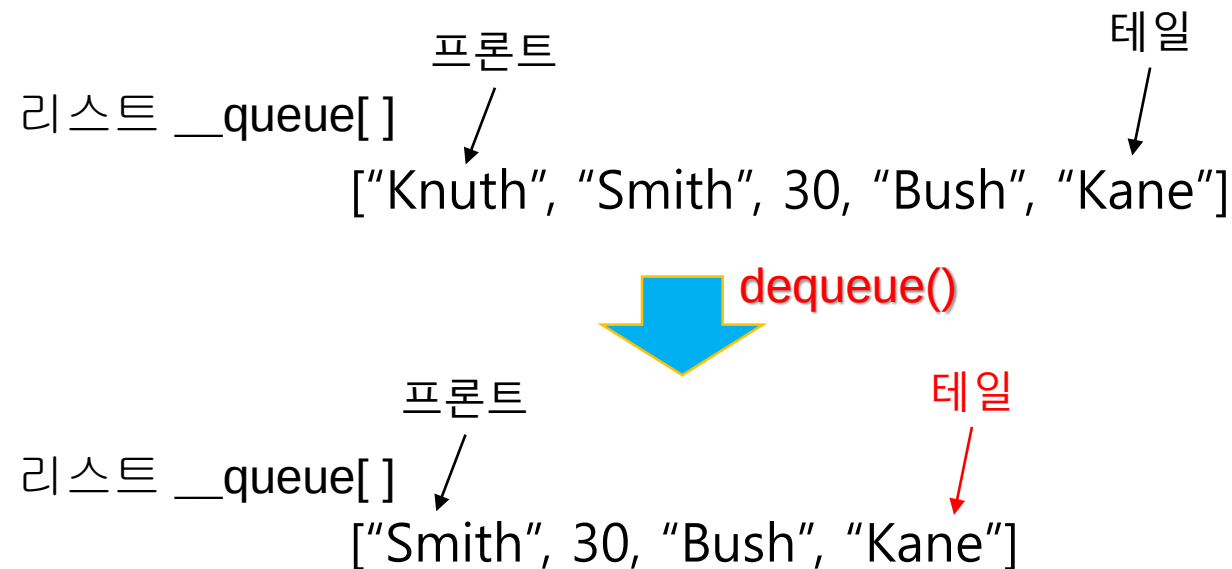
```
def enqueue(self, x):
    self.__queue.append(x)
```



리스트로 만든 큐에 원소를 삽입하는 예

삭제(Deletion) : dequeue

```
def dequeue(self):  
    return self.__queue.pop(0) # .pop(0): 리스트의 첫 원소를 삭제한 후 원소 리턴
```



배열 큐에서 원소를 삭제하는 예

기타 작업

- 큐의 맨 앞 요소 알려주기

```
def front(self):  
    return self.__queue[0]
```

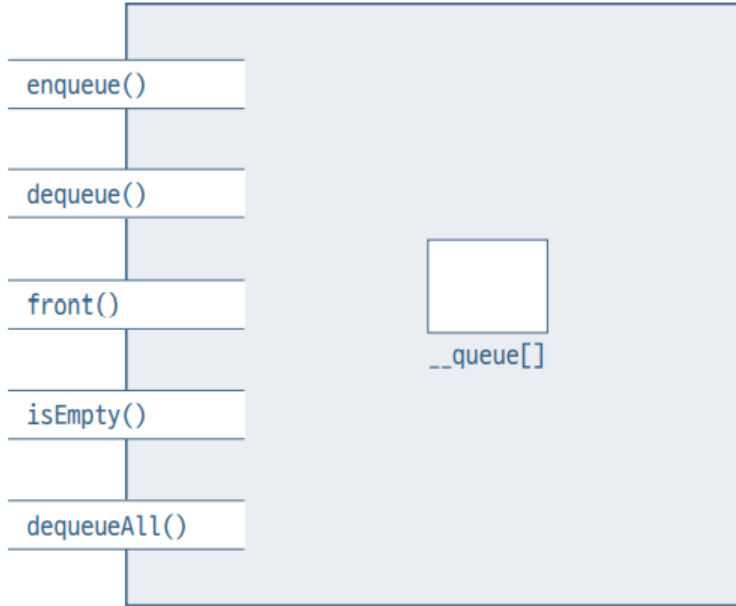
- 큐가 비어 있는지 확인하기

```
def isEmpty(self) -> bool:  
    return (len(self.__queue) == 0);
```

- 큐 비우기

```
def dequeueAll(self):  
    self.__queue = []    # 또는 self.__queue.clear()
```

파이썬 구현



```

class ListQueue:
    def __init__(self):
        self.__queue = []

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop(0) # pop(0): 리스트의 첫 원소 삭제 후 리턴

    def front(self):
        if self.isEmpty():
            return None
        else:
            return self.__queue[0]

    def isEmpty(self) -> bool:
        return (len(self.__queue) == 0)

    def dequeueAll(self):
        self.__queue.clear()

    def printQueue(self):
        print("Queue from front:", end=' ')
        for i in range(len(self.__queue)):
            print()
    
```

리스트 큐 사용 예

- 리스트 큐 사용 예

```
q1 = ListQueue()  
q1.enqueue("Mon")  
q1.enqueue("Tue")  
q1.enqueue(1234)  
q1.enqueue("Wed")  
q1.dequeue()  
q1.enqueue('aaa')  
q1.printQueue()
```

Queue from front: Tue 1234 Wed aaa

- 리스트 큐 사용 예

Queue from front: Tue 1234 wed aaa

선형 큐



← Animation

```
queue.enqueue(5)
```

```
queue.enqueue(2)
```

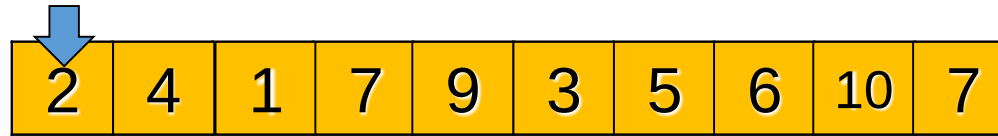
```
queue.enqueue(7)
```

```
queueFront ← queue.front()
```

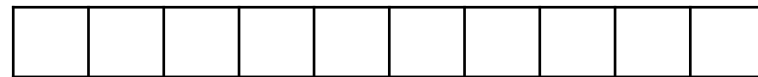
```
queueFront ← queue.dequeue()
```

```
queueFront ← queue.dequeue()
```

선형 큐



← Animation



front=tail

enqueue(2)



front

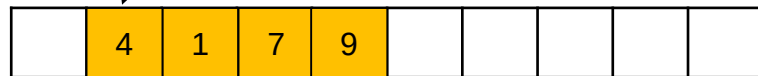
tail

enqueue(4)~ enqueue(9)



front

dequeue()



dequeue()

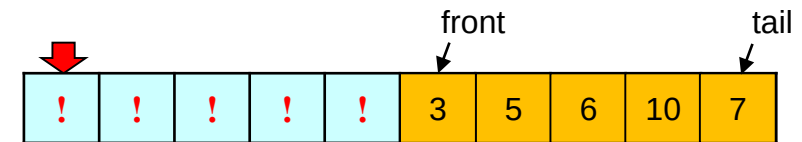
front

tail



enqueue(7)

tail

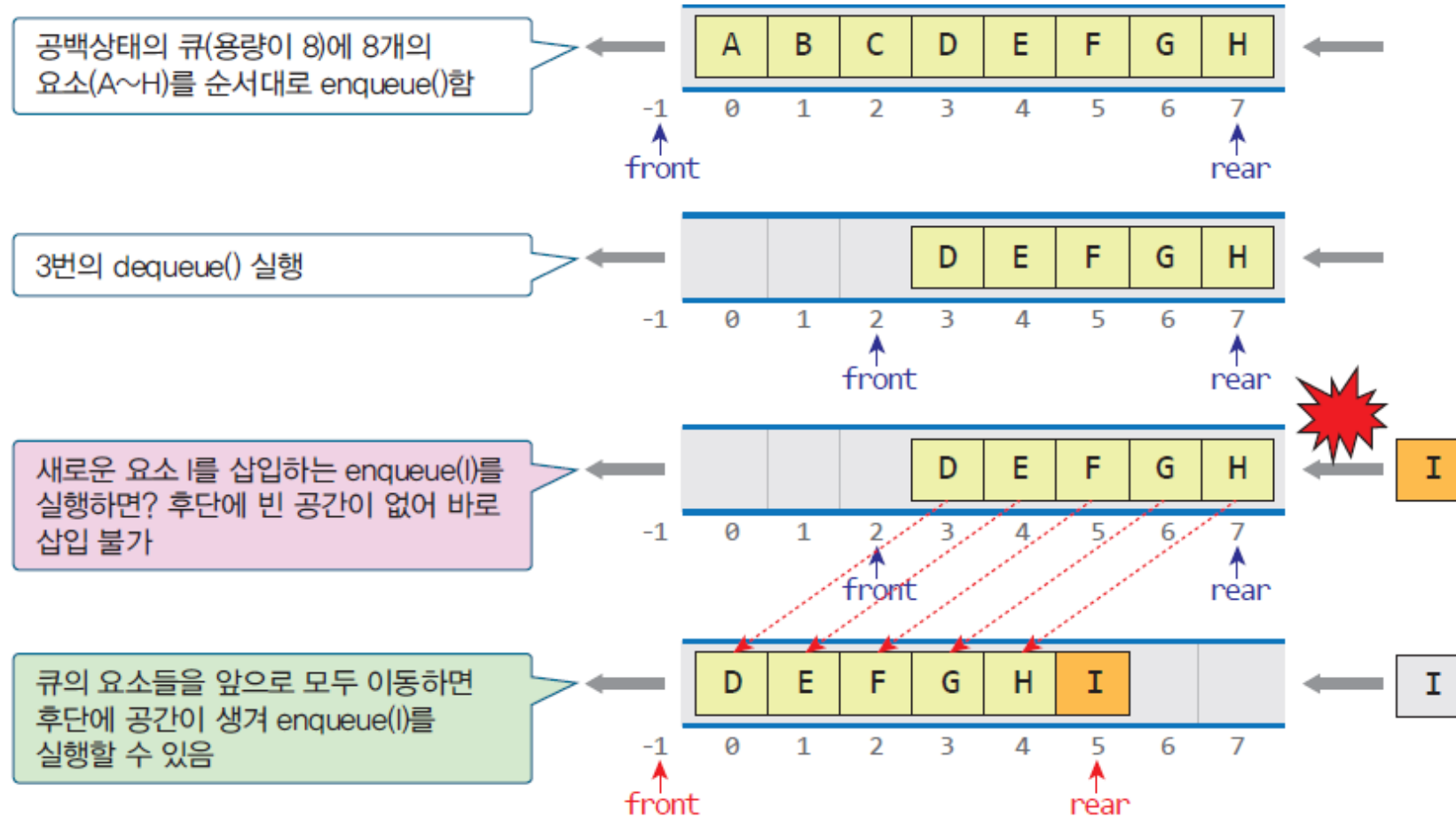


enqueue(9)

Queue full?

- 단순히 구현하면 공간이 있는데도 큐가 꽉 찬 것처럼 보인다

선형 큐



- 원형 큐(circular queue)로 구현하면 이 문제는 해결된다

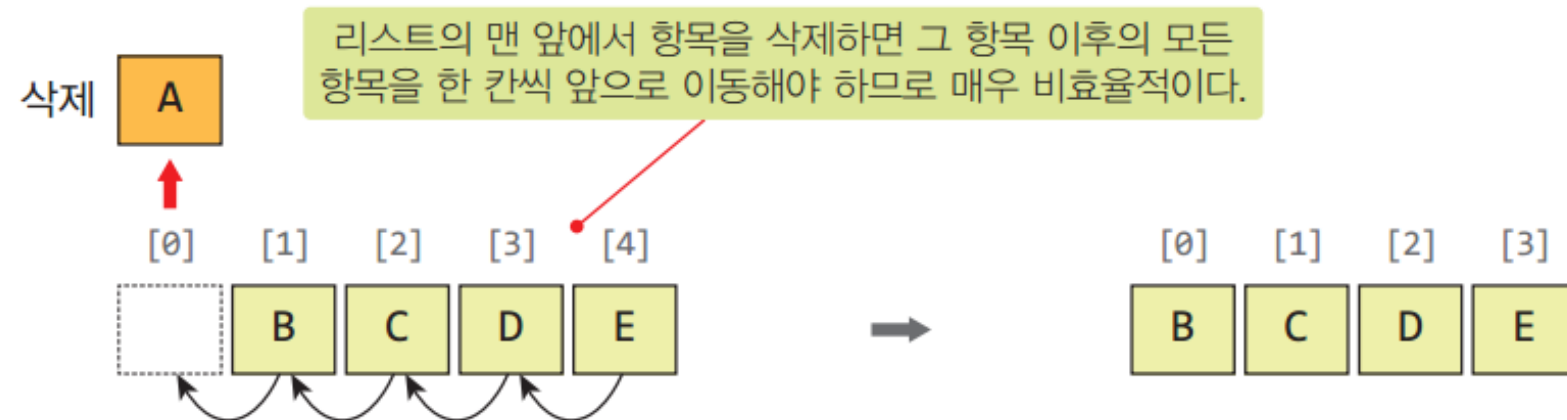
선형큐는 비효율적

enqueue(item): 삽입 연산

```
def enqueue(item):
    items.append(item)          # 리스트의 맨 뒤에 items 추가
```

dequeue(): 삭제 연산

```
def dequeue():
    if not isEmpty():          # 공백상태가 아니면
        return items.pop(0)    # 맨 앞 항목을 꺼내서 반환
```

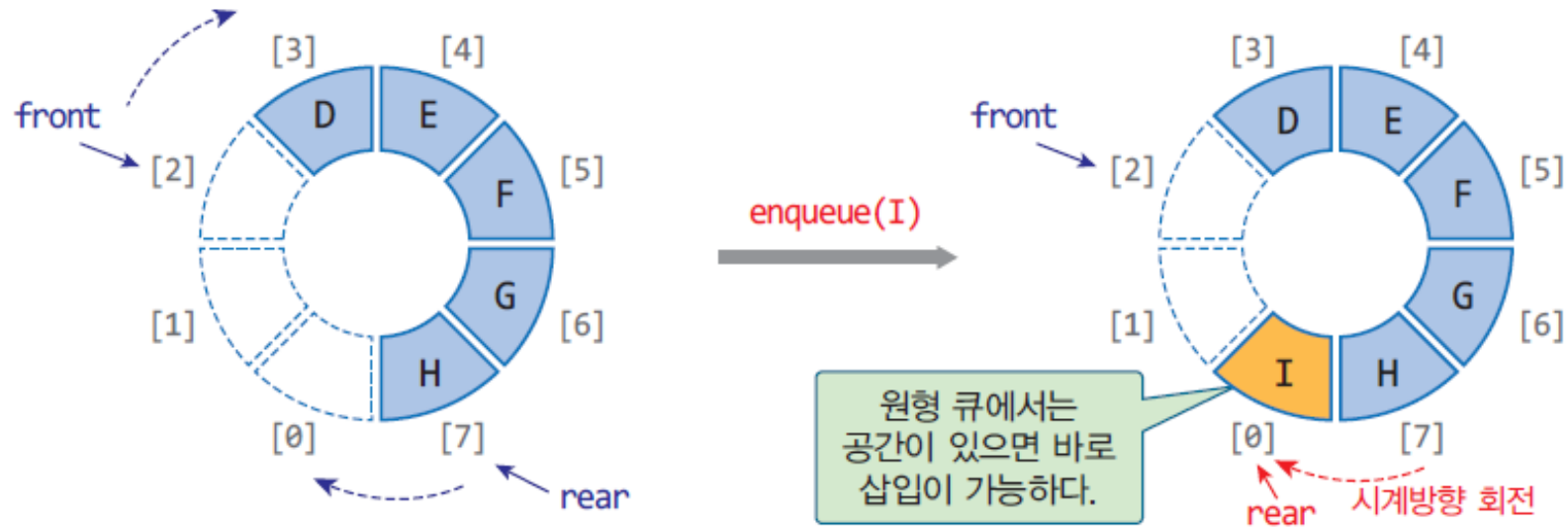
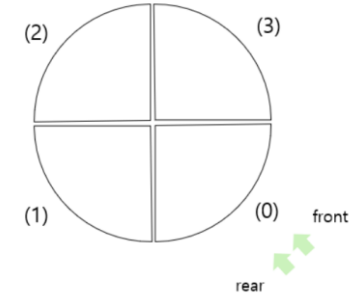


원형 큐(Circular Queue)

- 배열을 원형으로 사용 인덱스를 회전시키는 개념

front: 첫번째 요소 하나 앞의 인덱스

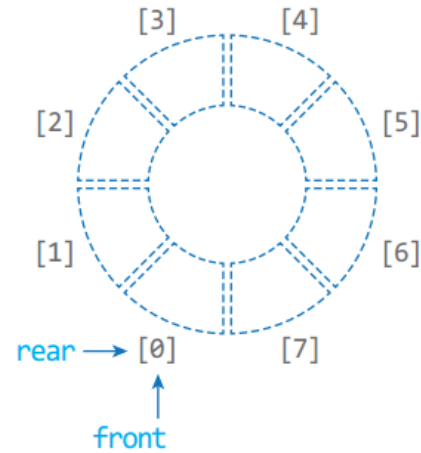
rear: 마지막 요소의 인덱스



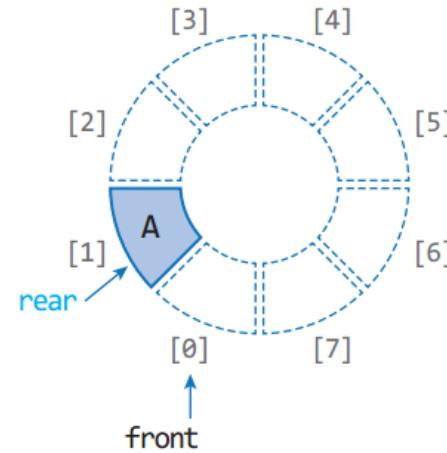
- 인덱스 회전(시계방향) 방법

- 전단 회전 : $\text{front} \leftarrow (\text{front} + 1) \% \text{capacity}$
- 후단 회전 : $\text{rear} \leftarrow (\text{rear} + 1) \% \text{capacity}$

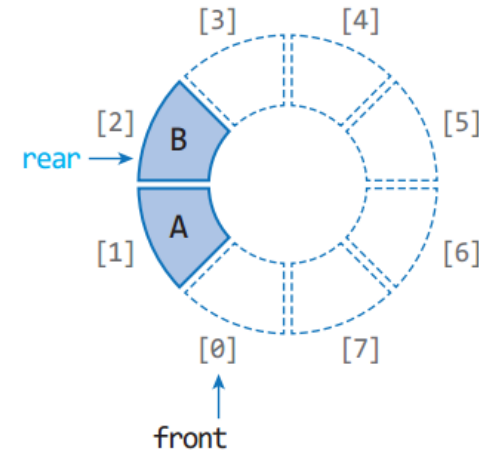
원형 큐의 삽입과 삭제 과정



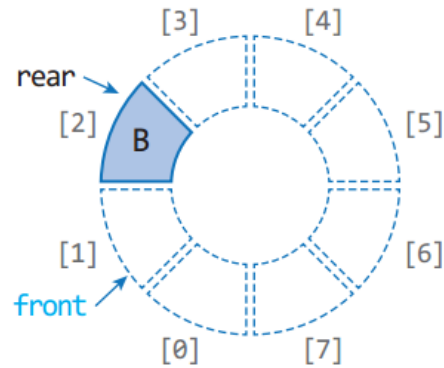
초기상태



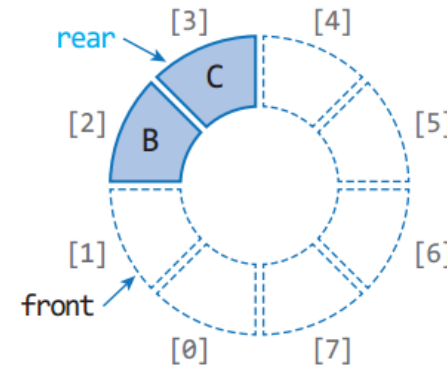
enqueue(A)



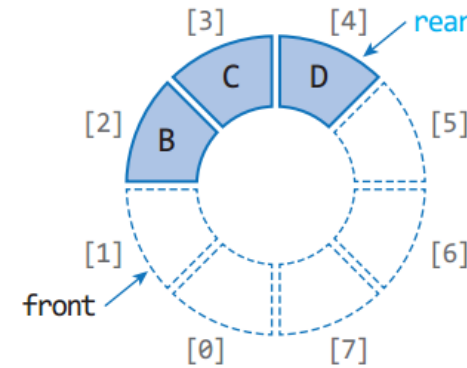
enqueue(B)



dequeue()



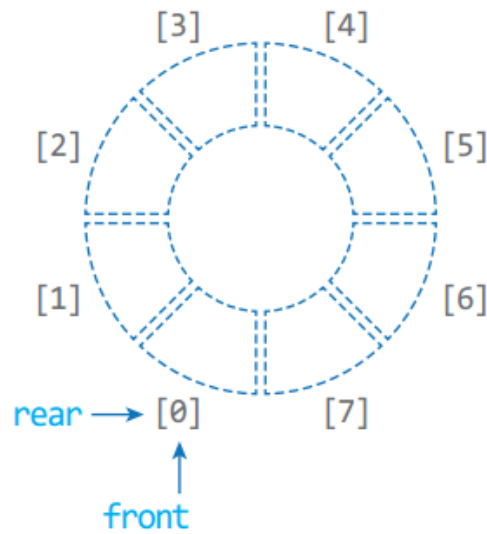
enqueue(C)



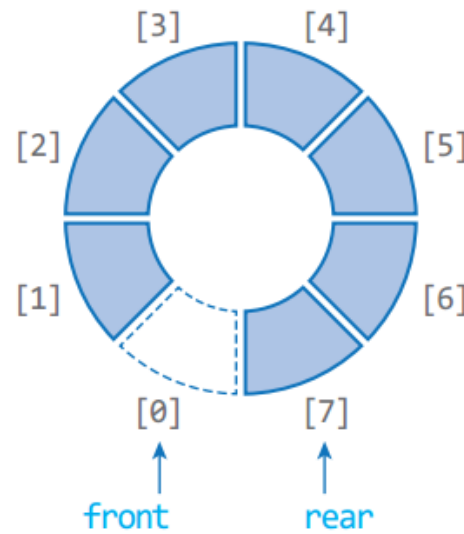
enqueue(D)

공백 상태와 포화 상태

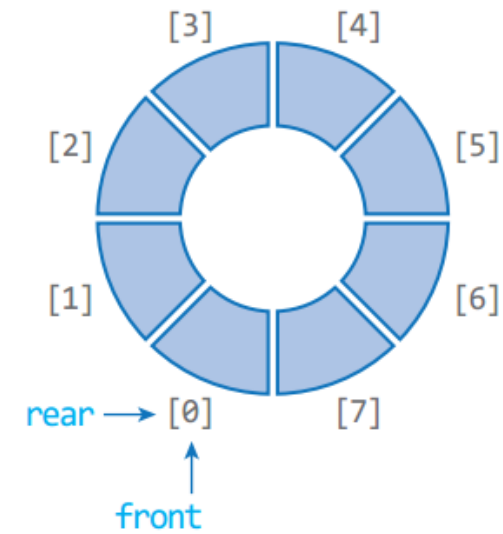
- 공백 상태: $front == rear$
- 포화 상태: $front \% M == (rear + 1) \% M$
- 공백 상태와 포화 상태를 구별 방법은?
 - 하나의 공간은 항상 비워둠



(a) 공백 상태



(c) 포화 상태



(b) 오류 상태

원형 큐의 구현

- 원형 큐 클래스

```
MAX_QSIZE = 10                                # 원형 큐의 크기
class CircularQueue :
    def __init__( self ) :                      # CircularQueue 생성자
        self.front = 0                         # 큐의 전단 위치
        self.rear = 0                          # 큐의 후단 위치
        self.items = [None] * MAX_QSIZE       # 항목 저장용 리스트 [None,None,...]

    def isEmpty( self ) : return self.front == self.rear
    def isFull( self ) : return self.front == (self.rear+1)%MAX_QSIZE
    def clear( self ) : self.front = self.rear
```

원형 큐의 구현

- 원형 큐 연산

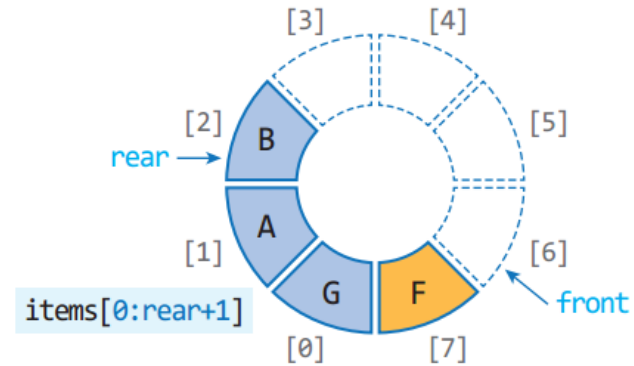
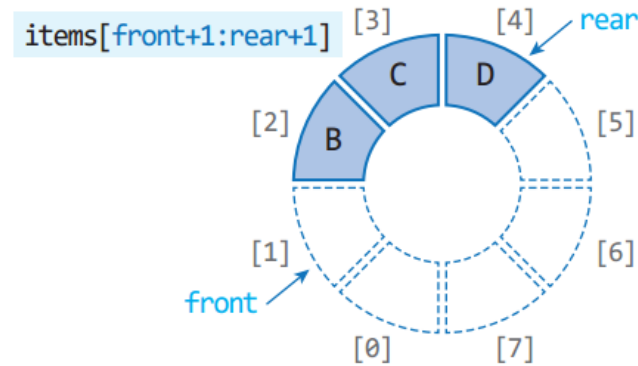
```
def enqueue( self, item ):  
    if not self.isFull():  
        self.rear = (self.rear+1)% MAX_QSIZE  
        self.items[self.rear] = item  
  
def dequeue( self ):  
    if not self.isEmpty():  
        self.front = (self.front+1)% MAX_QSIZE  
        return self.items[self.front]  
  
def peek( self ):  
    if not self.isEmpty():  
        return self.items[(self.front + 1) % MAX_QSIZE]  
  
def size( self ) :  
    return (self.rear - self.front + MAX_QSIZE) % MAX_QSIZE
```


원형 큐의 구현

- 원형 큐 출력

```

def display( self ):
    out = []
    if self.front < self.rear :
        out = self.items[self.front+1:self.rear+1]        # 슬라이싱
    else:
        out = self.items[self.front+1:MAX_QSIZE] W        # 다음 줄에 계속...
        + self.items[0:self.rear+1]                        # 슬라이싱
    print("[f=%s,r=%d] ==>"%(self.front, self.rear), out)
    
```



items[front+1:MAX_QSIZE]

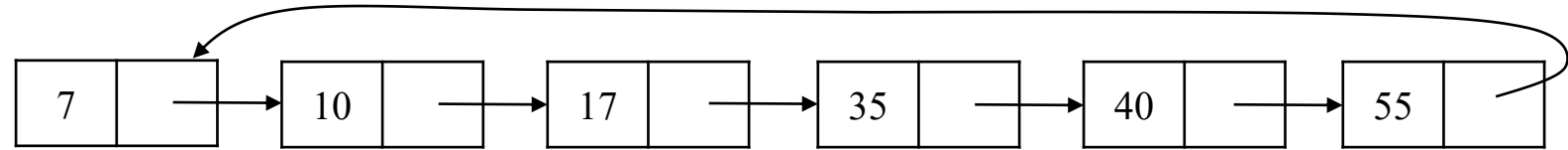
원형 큐 테스트

```
q = CircularQueue()
for i in range(8): q.enqueue(i)
q.display()
for i in range(5): q.dequeue();
q.display()
for i in range(8,14): q.enqueue(i)
q.display()
```

원형큐 만들기 (MAX_QSIZE=10)
0, 1, ... 7 삽입(f=0, r=8)
원형 큐에서 구현한 print()호출
5번 삭제 (f=5, r=8)
8, 9, ... 13 삽입 (f=5, r=4)

[f=0, r=8]	=>	[0, 1, 2, 3, 4, 5, 6, 7]
[f=5, r=8]	=>	[5, 6, 7]
[f=5, r=4]	=>	[5, 6, 7, 8, 9, 10, 11, 12, 13]

연결 리스트 큐 객체 구조



연결 리스트를 이용한 큐의 예

필드:

`__queue`

◀ 큐 원소들이 저장되는 리스트

작업:

`enqueue()`

◀ 큐의 맨 뒤에 원소를 삽입한다

`dequeue()`

◀ 큐의 맨 앞에 있는 원소를 알려주고 삭제한다

`front()`

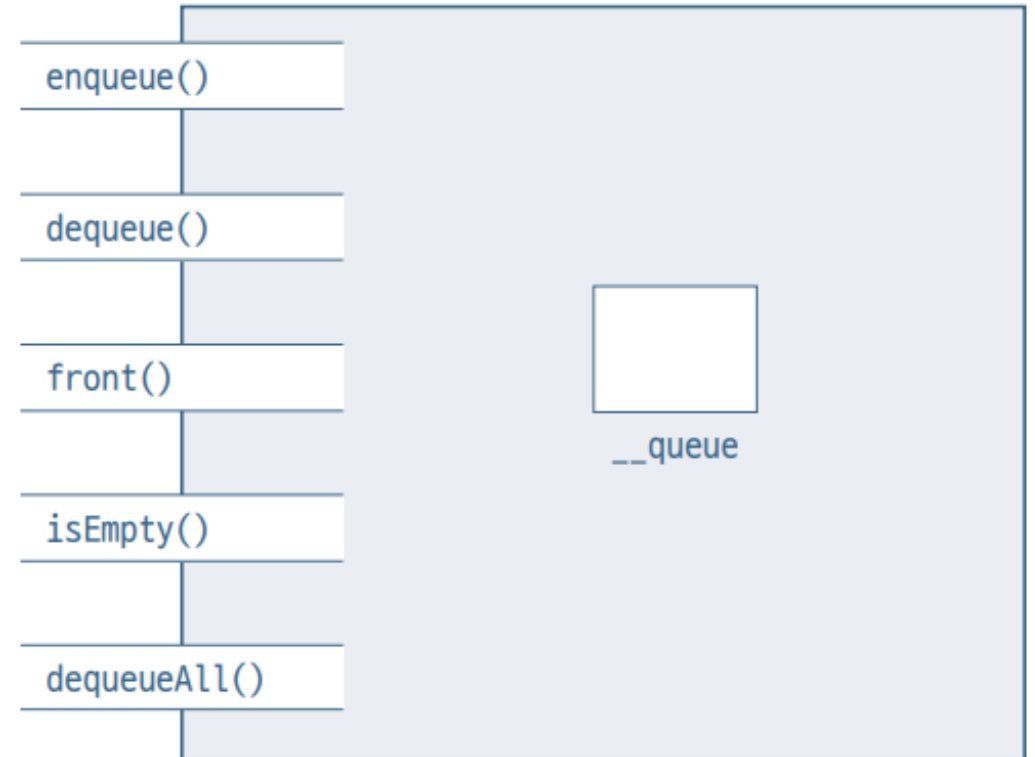
◀ 큐의 맨 앞에 있는 원소를 알려준다

`isEmpty()`

◀ 큐가 빈 큐인지를 알려준다

`dequeueAll()`

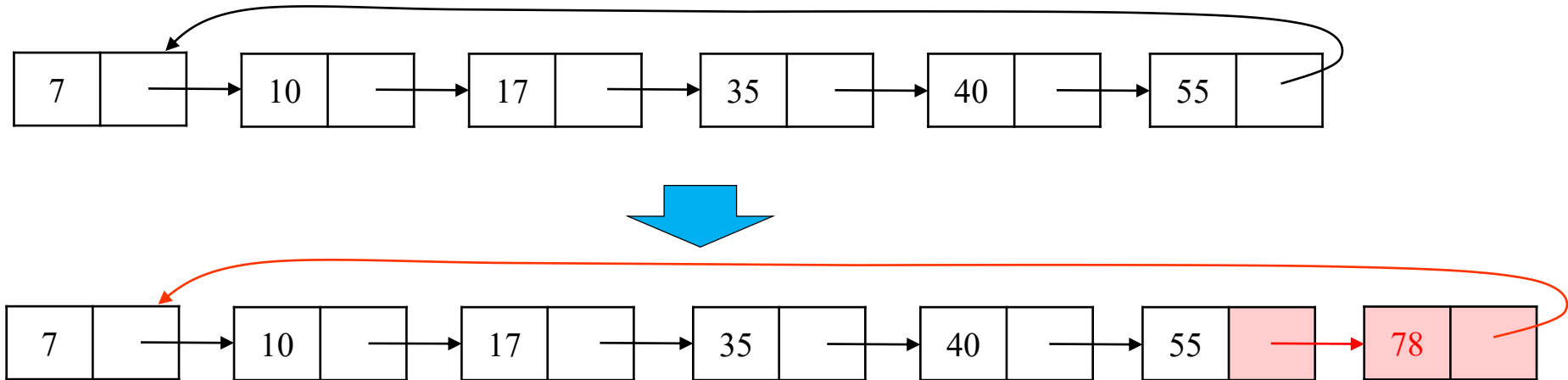
◀ 큐를 깨끗이 청소한다



연결 리스트 큐의 객체 구조

삽입(insertion) : enqueue

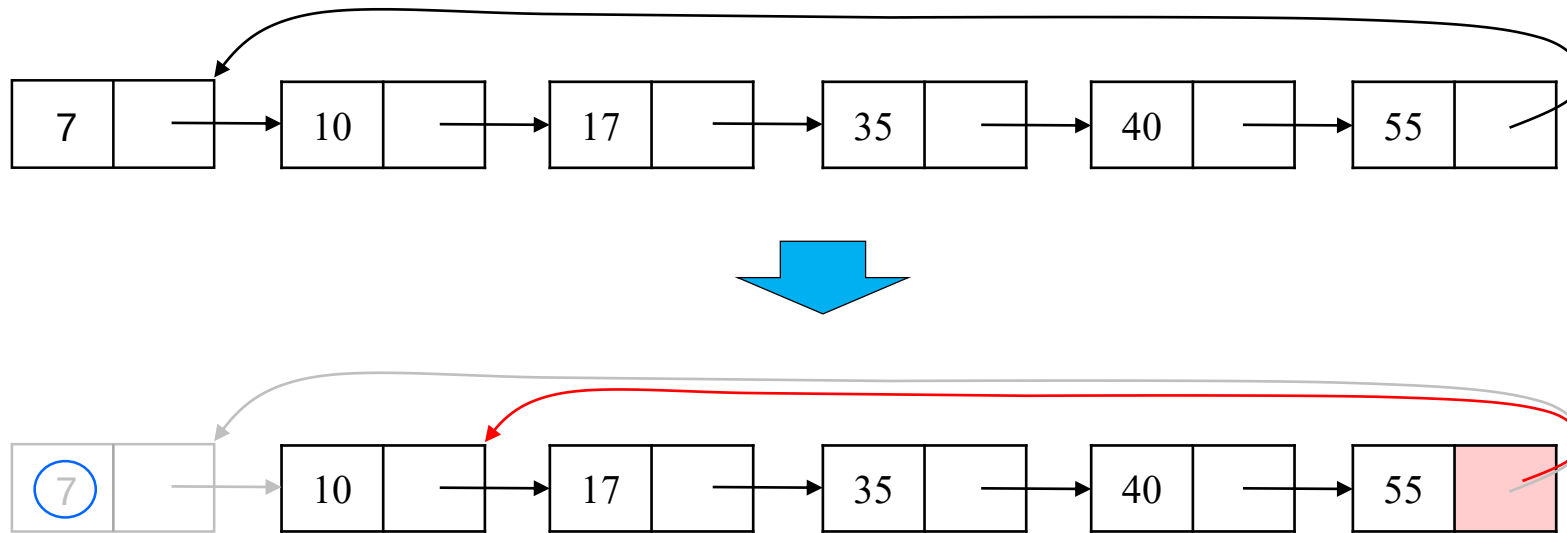
```
def enqueue(self, x):
    self.__queue.append(x)
```



원형 연결 리스트로 구현된 큐에 원소가 삽입되는 예

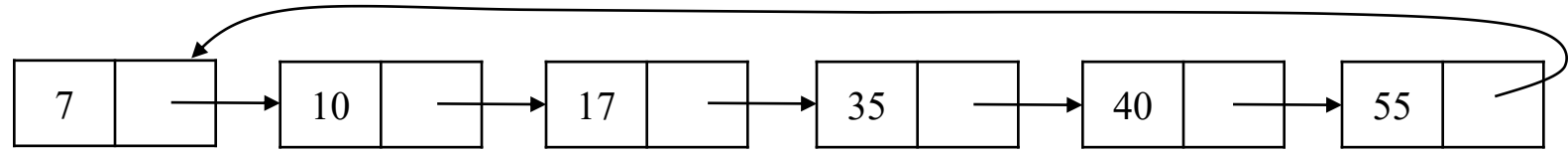
삭제(Deletion) : dequeue

```
def dequeue(self):
    return self.__queue.pop(0) # .pop(0): 리스트의 첫 원소를 삭제한 후 원소 리턴
```



원형 연결 리스트로 구현된 큐에서 삭제하는 예

기타 작업



- 큐의 맨 앞 요소 알려주기

```

def front(self):
    return self.__queue.get(0)    # .get(0): 리스트의 첫 원소 리턴
    
```

- 큐가 비어 있는지 확인하기

```

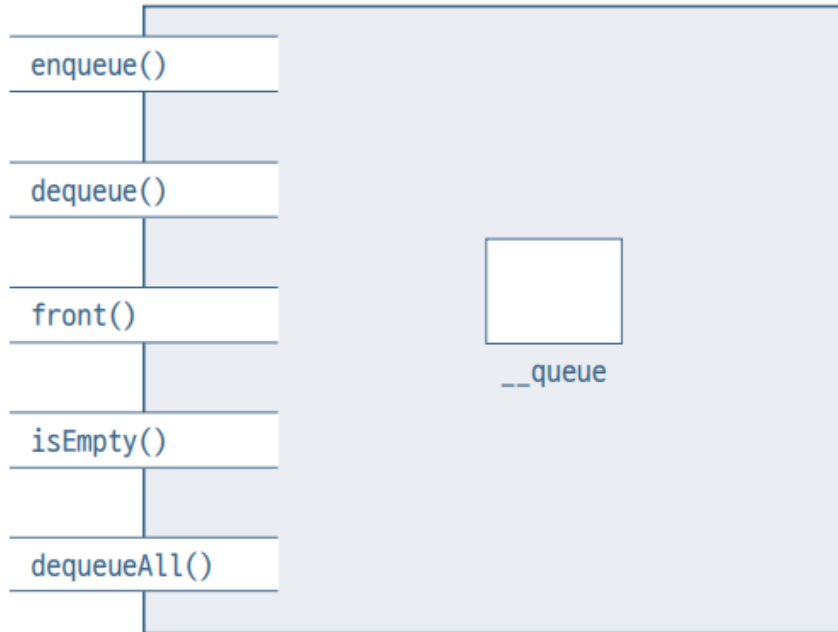
def isEmpty(self) -> bool:
    return self.__queue.isEmpty()
    
```

- 큐 비우기

```

def dequeueAll(self):
    self.__queue.clear()
    
```

파이썬 구현



원형 연결 리스트 큐 객체 구조

```

class LinkedQueue:
    def __init__(self):
        self.__queue = CircularLinkedList()

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop(0) # pop(0): 리스트의 첫 원소 삭제 후 리턴

    def front(self):
        return self.__queue.get(0) # .get(0): 리스트의 첫 원소 리턴

    def isEmpty(self) -> bool:
        return (self.__queue.isEmpty())

    def dequeueAll(self):
        self.__queue.clear()

    def printQueue(self):
        print("Queue from front:", end=' ')
        for i in range(self.__queue.size()):
            print(self.__queue.get(i), end=' ')
        print()
    
```

클래스 ListQueue 사용 예

- 리스트 큐 사용 예

```
q1 = LinkedQueue()
q1.enqueue("Mon")
q1.enqueue("Tue")
q1.enqueue(1234)
q1.enqueue("Wed")
q1.dequeue()
q1.enqueue('aaa')
q1.printQueue()
```

- 리스트 큐 사용 예

```
Queue from front: Tue 1234 wed aaa
```

Palindrome 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

- palindrome(좌우동형) 문자열 : 앞에서부터 읽으나 뒤에서부터 읽으나 같은 문자열
- 예
abbcbbba : 좌우동형 o
abb : 좌우동형 x

출력

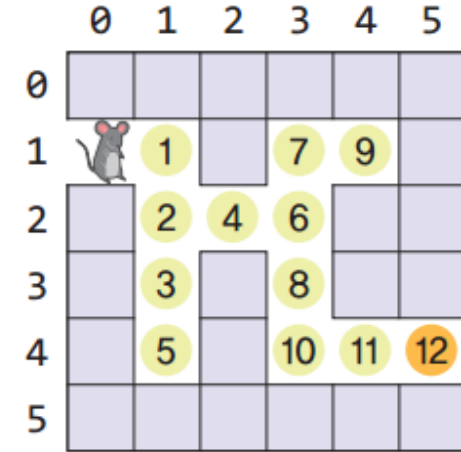
```
Palindrome Check!
Is lioninoil Palindrome?: true
```


Palindrome 체크

```
def is_palindrome(input_string):  
    token_list = list(input_string)  
    flag = True  
    while len(token_list) > 1 and flag == True:  
        first_char = token_list.pop(0)  
        last_char = token_list.pop(-1)  
        if first_char != last_char:  
            flag = False  
    return flag  
  
print(is_palindrome('racecar'))  
print(is_palindrome('raddar'))  
print(is_palindrome('wasitacatisaw'))
```

미로 탐색: 너비우선탐색

위치표시: (x, y)

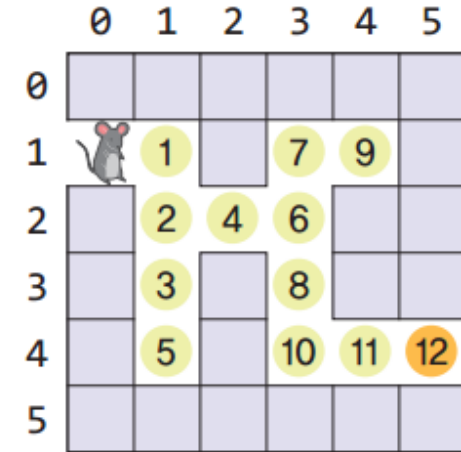


1				
1	2			
2				
2	3	4		
3	4			

3	4	5		
4	5			
4	5	6		
5	6			
6				

미로 탐색: 너비우선탐색

위치표시: (x, y)



6	7	8		
7	8			
7	8	9		
8	9			
8	9	10		

9	10			
10				
10	11			
11	12			
12				

너비우선탐색 알고리즘

```
def BFS() :                                # 너비우선탐색 함수
    que = CircularQueue()
    que.enqueue((0,1))
    print('BFS: ')                          # 출력을 'BFS'로 변경

    while not que.isEmpty():
        here = que.dequeue()
        print(here, end='->')
        x,y = here
        if (map[y][x] == 'x') : return True
        else :
            map[y][x] = '.'
            if isValidPos(x, y - 1) : que.enqueue((x, y - 1))    # 상
            if isValidPos(x, y + 1) : que.enqueue((x, y + 1))    # 하
            if isValidPos(x - 1, y) : que.enqueue((x - 1, y))    # 좌
            if isValidPos(x + 1, y) : que.enqueue((x + 1, y))    # 우
    return False
```

너비우선탐색 알고리즘 테스트

```

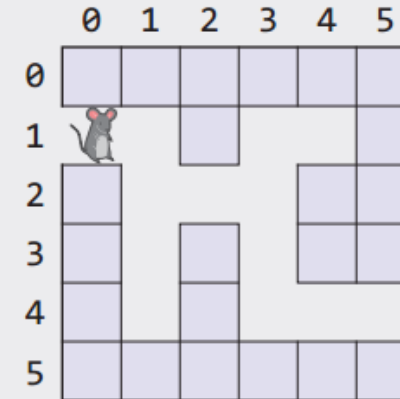
map = [ [ '1', '1', '1', '1', '1', '1' ],
        [ 'e', '0', '1', '0', '0', '1' ],
        [ '1', '0', '0', '0', '1', '1' ],
        [ '1', '0', '1', '0', '1', '1' ],
        [ '1', '0', '1', '0', '0', 'x' ],
        [ '1', '1', '1', '1', '1', '1' ] ]
    
```

MAZE_SIZE = 6

result = BFS()

if result : print(' --> 미로탐색 성공')

else : print(' --> 미로탐색 실패')

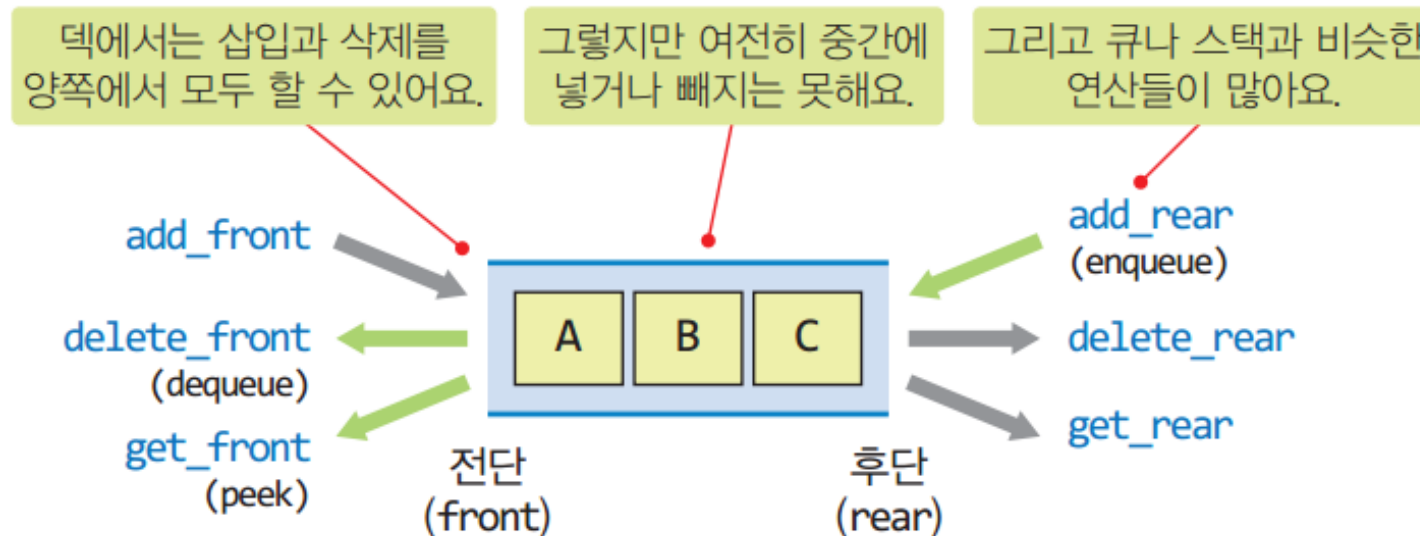


너비우선탐색 과정

(0, 1)→(1, 1)→(1, 2)→(1, 3)→(2, 2)→(1, 4)→(3, 2)→(3, 1)→(3, 3)→(4, 1)→
 (3, 4)→(4, 4)→(5, 4)→ --> 미로탐색 성공

덱(Deque)

- double-ended queue
- 전단(front)와 후단(rear)에서 모두 삽입과 삭제가 가능한 큐
- 스택이나 큐보다 입출력이 자유로운 자료구조
- 덱은 양쪽 끝에서 삽입과 삭제를 허용하는 자료구조로서 스택과 큐 자료구조를 혼합한 자료구조
- 덱은 스크롤, 문서 편집기의 undo 연산, 웹 브라우저의 방문 기록 등에 사용



Deque ADT

데이터: 전단과 후단을 통한 접근을 허용하는 항목들의 모임
연산

- Deque(): 비어 있는 새로운 덱을 만든다.
- isEmpty(): 덱이 비어있으면 True를 아니면 False를 반환한다.
- addFront(x): 항목 x를 덱의 맨 앞에 추가한다.
- deleteFront(): 맨 앞의 항목을 꺼내서 반환한다.
- getFront(): 맨 앞의 항목을 꺼내지 않고 반환한다.
- addRear(x): 항목 x를 덱의 맨 뒤에 추가한다.
- deleteRear(): 맨 뒤의 항목을 꺼내서 반환한다.
- getRear(): 맨 뒤의 항목을 꺼내지 않고 반환한다.
- isFull(): 덱이 가득 차 있으면 True를 아니면 False를 반환한다.
- size(): 덱의 모든 항목들의 개수를 반환한다.
- clear(): 덱을 공백상태로 만든다.

Deque의 구현

- 원형 큐를 상속하여 원형 덱 클래스를 구현

```
class CircularDeque(CircularQueue) :    # CircularQueue에서 상속
```

- 덱의 생성자 (상속되지 않음)

```
def __init__( self ) :                # CircularDeque의 생성자  
    super().__init__()                # 부모 클래스의 생성자를 호출함
```

front, rear, items와 같은 멤버 변수는 추가로 선언하지 않음
자식클래스에서 부모를 부르는 함수가 super()

- 재 사용 멤버들: isEmpty, isFull, size, clear
- 인터페이스 변경 멤버들

```
def addRear( self, item ) : self.enqueue(item )    # enqueue 호출  
def deleteFront( self ) : return self.dequeue()    # 반환에 주의  
def getFront( self ) : return self.peek()          # 반환에 주의
```


Deque의 구현

- 추가로 구현할 메소드

```
def addFront( self, item ):                # 새로운 기능: 전단 삽입
    if not self.isFull():
        self.items[self.front] = item        # 항목 저장
        self.front = self.front - 1          # 반시계 방향으로 회전
        if self.front < 0 : self.front = MAX_QSIZE - 1

def deleteRear( self ):                    # 새로운 기능: 후단 삭제
    if not self.isEmpty():
        item = self.items[self.rear];        # 항목 복사
        self.rear = self.rear - 1            # 반시계 방향으로 회전
        if self.rear < 0 : self.rear = MAX_QSIZE - 1
    return item                            # 항목 반환

def getRear( self ):                      # 새로운 기능: 후단 peek
    return self.items[self.rear]
```

Deque 테스트

```

dq = CircularDeque()
for i in range(9):
    if i%2==0 : dq.addRear(i)
    else : dq.addFront(i)
dq.display()
for i in range(2): dq.deleteFront()
for i in range(3): dq.deleteRear()
dq.display()
for i in range(9,14): dq.addFront(i)
dq.display()
    
```

덱 객체 생성. f=r=0
 # i : 0, 1, 2, ... 8
 # 짝수는 후단에 삽입:
 # 홀수는 전단에 삽입
 # front=6, rear=5
 # 전단에서 두 번의 삭제: f=8
 # 후단에서 세 번의 삭제: r=2
 # i : 9, 10, ... 13 : f=3

[f=6, r=5]	=>	[7, 5, 3, 1, 0, 2, 4, 6, 8]
[f=8, r=2]	=>	[3, 1, 0, 2]
[f=3, r=2]	=>	[13, 12, 11, 10, 9, 3, 1, 0, 2]

파이썬에서 큐와 덱 사용하기

방법 1) queue 모듈의 Queue 사용하기

```

import queue                                # 파이썬의 queue 모듈 포함
q = queue.Queue(maxsize=20)                # 큐 객체 생성(최대크기 20)
    
```

연산	ArrayQueue	queue.Queue
삽입/삭제	enqueue(), dequeue()	put(), get()
공백/포화 상태 검사	isEmpty(), isFull()	empty(), full()
전단 들여다보기	peek()	제공하지 않음

파이썬에서 큐와 덱 사용하기

방법 2) collections 모듈의 deque 클래스 사용하기

```
import collections      # 파이썬의 collections 모듈 포함
dq = collections.deque() # 용량이 무한대인 덱 객체 생성
```

연산	CircularDeque	collections.deque
전단 삽입/삭제	addFront(), deleteFront()	appendleft(), popleft()
후단 삽입/삭제	addRear(), deleteRear()	append(), pop()
공백 상태 검사	isEmpty()	dq (예: if dq : ...)
포화 상태 검사	isFull()	의미 없음
들여다보기	getFront(), getRear()	제공하지 않음

Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://drive.google.com/drive/folders/1XXNIpQFYNRJ3xRIMYg7kz6zNlOATMqZm?usp=sharing>